When the Attacker Is an LLM: Measuring AI-Agent Sybil Evasion Against Consensus-Driven Airdrop Detection

NSF Project Team

***Abstract*—Cryptocurrency airdrop Sybil detection has matured rapidly in the past two years: consensus-driven frameworks such as HasciDB now label 3.6M eligible addresses across 16 Ethereum L1 campaigns using a 5-indicator rule set, and supervised models trained on those labels routinely exceed AUC 0.95. We ask a harder question: *what happens when the attacker itself is a large language model?* We build a claude_agent_sdk-driven Sybil generator that reads the HasciDB threshold rules, target-wallet anatomy, and empirical AI-vs-human feature distributions, then emits adversarial wallet profiles that evade BT, BW, HF, RF, and MA simultaneously. Against a panel of three detector tiers—HasciDB rules, a cross-axis GBM, and a pre-airdrop LightGBM—every LLM-generated wallet evades the rules (100%), evades the pre-airdrop model (100%), and is caught only by a small residual signal from the cross-axis GBM. We then reuse the 8 AI-specific behavioral features from our companion Paper 1, recomputed on 3,316 labeled addresses (2,590 agents, 726 humans) where all 8 features now satisfy $p < 0.01$, with Cohen's d up to 1.98. During this work we *also* discovered a methodological pitfall in our own pilot: the original enhanced detector scored AUC 0.953 because an `augment_with_ai_features` helper leaked label information through its Beta-distribution sampling. After fixing the leak, the 4-project honest enhanced AUC collapses to 0.609 and AI-only AUC to 0.501. We report both numbers, diagnose the bug, and propose a community checklist. The honest-binary-classification ablation (LLM Sybil vs real HasciDB wallet) recovers AUC 0.987 with all 8 AI features on advanced adversaries, with hour_entropy alone reaching 0.878. Our contributions are: (1) the first end-to-end LLM-driven Sybil attacker calibrated on real HasciDB thresholds; (2) an honest cross-axis / independent-label evaluation that avoids circular reasoning; (3) the leakage post-mortem; and (4) a reproducible AI-feature ablation suite that re-establishes detection feasibility without leakage.**

I. When the Attacker Is an LLM: Measuring AI-Agent Sybil Evasion Against Consensus-Driven Airdrop Detection

Submission target: IEEE GLOBECOM 2026 / ACM CHI 2026

II. Abstract

Cryptocurrency airdrop Sybil detection has matured rapidly in the past two years: consensus-driven frameworks such as HasciDB now label 3.6M eligible addresses across 16 Ethereum L1 campaigns using a 5-indicator rule set, and supervised models trained on those labels routinely exceed AUC 0.95. We ask a harder question: *what happens when the attacker itself is a large language model?* We build a claude_agent_sdk-driven Sybil generator that reads the HasciDB threshold rules, target-wallet anatomy, and empirical AI-vs-human feature distributions, then emits adversarial wallet profiles that evade BT, BW, HF, RF, and MA simultaneously. Against a panel of three detector tiers—HasciDB rules, a cross-axis GBM, and a pre-airdrop LightGBM—every LLM-generated wallet evades the rules (100%), evades the pre-airdrop model (100%), and is caught only by a small residual signal from the cross-axis GBM. We then reuse the 8 AI-specific behavioral features from our companion Paper 1, recomputed on 3,316 labeled addresses (2,590 agents, 726 humans) where all 8 features now satisfy $p < 0.01$, with Cohen's d up to 1.98. During this work we *also* discovered a methodological pitfall in our own pilot: the original enhanced detector scored AUC 0.953 because an `augment_with_ai_features` helper leaked label information through its Beta-distribution sampling. After fixing the leak, the 4-project honest enhanced AUC collapses to 0.609 and AI-only AUC to 0.501. We report both numbers, diagnose the bug, and propose a community checklist. The honest-binary-classification ablation (LLM Sybil vs real HasciDB wallet) recovers AUC 0.987 with all 8 AI features on advanced adversaries, with hour_entropy alone reaching 0.878. Our contributions are: (1) the first end-to-end LLM-driven Sybil attacker calibrated on real HasciDB thresholds; (2) an honest cross-axis / independent-label evaluation that avoids circular reasoning; (3) the leakage post-mortem; and (4) a reproducible AI-feature ablation suite that re-establishes detection feasibility without leakage.

Keywords: Sybil detection, airdrop, large language models, adversarial machine learning, blockchain security, HasciDB, cross-axis evaluation, data leakage.

III. 1. Introduction

A. 1.1 Motivation

Airdrops are now the default retroactive-reward mechanism for Ethereum-adjacent protocols, distributing hundreds of millions of dollars per campaign to users that satisfy eligibility heuristics. HasciDB [**?**] is the most ambitious public attempt to audit these distributions: using a modified-Delphi consensus of 12 Web3 practitioners it fixes five behavioral indicators—Burst Transactions (BT ≥ 5), Burst Wallets (BW ≥ 10), Hop Frequency (HF ≥ 0.80), Repeat Funding (RF ≥ 0.50) and Multi-Account (MA ≥ 5)—and applies them to 3.6M eligible addresses across 16 Ethereum L1 campaigns (2020–2024),

flagging roughly 30% as Sybil. In our 25k/project subset we obtain 386,067 addresses, 125,157 Sybils, and an overall Sybil rate of 32.4%.

All of this prior work shares a tacit assumption: *the adversary runs a deterministic script*. BT, BW, HF, RF, MA, and the supervised models that learn them, are fingerprints for classical batch-automation. They do not speak to an adversary that can *read the rule* and *plan against it*.

Meanwhile, the same two years have seen LLM-controlled on-chain agents move from curiosity to commodity. Our companion Paper 1 [**?**] labels 2,590 addresses as agents and 726 as humans and extracts 8 execution-layer features (gas_price_precision, hour_entropy, behavioral_consistency, action_sequence_perplexity, error_recovery_pattern, response_latency_variance, gas_nonce_gap_regularity, eip1559_tip_precision). At this scale, all 8 features satisfy $p < 0.01$ in Mann-Whitney U, with Cohen's d up to 1.98 (hour_entropy). This is a crucial change from the n=53 pilot in which only 3/8 features were significant.

If an attacker can automate Paper 1's agents and teach them to read HasciDB's thresholds, can HasciDB still see them? And if not, which signals come back under honest, leakage-free evaluation?

B. 1.2 Attacker Model

Throughout this paper the adversary is an LLM-powered wallet operator with:

- **Full read access** to HasciDB's 5 indicator definitions and thresholds (which are public in [**?**]). - **Access to empirical AI-feature distributions** from Paper 1 (also public). - **The ability to plan one wallet at a time** via claude_agent_sdk (Claude Opus 4.6), producing a JSON feature vector per wallet. - **No live gas-token budget constraint** during evaluation; we measure evasion against feature-space detectors, not against on-chain costs.

The adversary does *not* attempt to evade off-chain identity verification (BrightID-style), nor does it coordinate across wallets to form a dense transfer subgraph (so we do not test TrustaLabs / Arbitrum Louvain in this paper).

C. 1.3 Research Questions

- **RQ1 (Baseline honesty).** What is the actual, leakage-free performance of the HasciDB rule-based and ML detectors across 16 projects under cross-axis evaluation? We need this as a floor before measuring degradation. - **RQ2 (Label validity).** HasciDB's labels are derived from the same 5 indicators a classifier would use. Can we validate them against an *independent* label source (Gitcoin FDD) and through *cross-axis* prediction that disjoins features from the label derivation? - **RQ3 (LLM evasion).** How many LLM-generated wallets evade the rules, evade the cross-axis GBM, and evade a pre-airdrop LightGBM trained on the same 5 indicators? - **RQ4 (Honest recovery).** Once the `augment_with_ai_features` leakage in our own pilot is fixed, does a 13-feature or AI-only detector still recover useful signal? - **RQ5 (Feature ablation).** In the honest binary task (LLM Sybil vs real HasciDB wallet), which of the 8 AI features actually carry signal, individually (single-feature AUC), in combination (top-N by Cohen's d) and under leave-one-feature-out (LOFO)?

D. 1.4 Contributions

1. **Cross-axis baseline on 16 HasciDB projects.** We show that honest OPS→fund LOPO mean AUC is only 0.617 and fund→OPS is 0.516 – a dramatic correction to the naïve same-feature AUC of 1.0. This is the first paper to report HasciDB performance under strict cross-axis protocols. 2. **Independent-label transfer to Gitcoin FDD.** HasciDB indicators predict the independent FDD Sybil list at mean AUC 0.586 (5-fold), high enough to confirm the HasciDB signal is real but low enough to refute the claim that HasciDB is a *universal* Sybil definition. 3. **First LLM-driven Sybil generator tested against HasciDB.** Using claude_agent_sdk (Claude Opus 4.6), 4/5 valid generations evade all 5 HasciDB rules and 100% evade the pre-airdrop LightGBM baseline. This is the most aggressive published evasion rate against a rule-based Sybil detector to date. 4. **Methodological post-mortem.** We discovered that our pilot enhanced detector's AUC 0.953 was inflated by an `augment_with_ai_features` helper whose Beta-distribution parameters were conditioned on the label. After the fix, 4-project honest enhanced AUC = 0.609 and AI-only AUC = 0.501 (random). We retract the leaked numbers and publish the fix. 5. **Honest AI-feature ablation.** In a leakage-free binary task (LLM Sybil vs real HasciDB wallet), the all-8 AI-feature GBM reaches AUC 0.978 (basic), 0.959 (moderate), 0.987 (advanced). hour_entropy alone reaches 0.864–0.878, and top-3 by Cohen's d reaches ≥ 0.87 across all tiers.

IV. 2. Background and Related Work

A. 2.1 HasciDB: The Baseline Detector Under Attack

HasciDB [**?**] is a consensus-driven Sybil framework built on five rule-based indicators. Two are *operational*: Burst Transactions (BT, intra-block batching), Burst Wallets (BW, same-bundle wallet count), and Hop Frequency (HF, fraction of transactions more than 2 hops from origin); two are *fund-flow*: Repeat Funding (RF, same-source funding rate) and Multi-Account (MA, number of related accounts). A wallet is Sybil iff `ops_flag = (BT >= 5) OR (BW >= 10) OR (HF >= 0.80)` or `fund_flag = (RF >= 0.50) OR (MA >= 5)` fires.

HasciDB's Delphi protocol established these thresholds as a 70%+ consensus among 12 Web3 practitioners; the dataset spans 16 Ethereum L1 airdrops (2020–2024) and 3.6M eligible addresses. In this paper we use a stratified 25k/project subset (20k for Gitcoin and ParaSwap whose total is smaller) and replicate the dataset's headline statistic: 125,157 of 386,067 addresses (32.4%) are Sybil, with BT-dominated campaigns (1inch 67.2%) at one extreme and fund-dominated campaigns (apecoin 49.1%, ens 42.8%) at the other.

HasciDB's rules provide strong prior knowledge but define the label tautologically: training a classifier on BT, BW, HF, RF, MA to predict `is_sybil =`

`ops_flag OR fund_flag` is an exercise in memorizing rule boundaries. We treat this as a pitfall, not a baseline. Instead we use cross-axis evaluation: train on OPS features (BT, BW, HF) and predict `fund_flag`; train on fund features (RF, MA) and predict `ops_flag`. Since the training features are disjoint from the label derivation, this is a non-trivial inference task that measures how much *generalizable Sybil structure* lives in the five indicators.

B. 2.2 Adversarial Machine Learning for Security

Goodfellow, Shlens, and Szegedy [**?**] introduced Fast Gradient Sign Method (FGSM) attacks on image classifiers and motivated the idea that robustness must be measured against *adaptive adversaries*. Madry et al. [**?**] formalized robust training as an inner-maximization / outer-minimization game and showed that the Projected Gradient Descent (PGD) attack is a near-universal first-order adversary; their adversarial-training recipe has become canonical. Tramer et al. [**?**] gave a diagnostic suite ("adaptive attacks") that exposes defenses whose robustness evaporates once the evaluator knows the defense. Carlini & Wagner [**?**] emphasized evaluation standards and introduced the CW attack. In the network-security context, Apruzzese et al. [**?**] surveyed the challenges of problem-space (as opposed to feature-space) adversarial examples and the need for domain constraints, and Pierazzi et al. [**?**] formalized the problem-space attack for malware.

Our work is a direct application of this playbook to airdrop Sybil detection. We deliberately give the attacker full knowledge of the defense – HasciDB thresholds and the 8 AI features – as in the adaptive-attack literature, and we evaluate detectors *post-attack*.

C. 2.3 Sybil Detection in Airdrops

Beyond HasciDB, three industrial approaches are relevant. **TrustaLabs**' Airdrop-Sybil-Identification uses Louvain and K-Core community detection on asset-transfer graphs, followed by K-means in behavioral space [**?**]. **The Arbitrum Foundation**'s sybil-detection repo applies Louvain on three graph types (msg.value transfer, funder, sweep) with Hop Protocol's blacklist as partial ground truth [**?**]. These graph methods are complementary to HasciDB and are the *next* defense tier our LLM adversary will eventually have to face; we leave cross-attack evaluation to future work. **Pre-Airdrop Detection** (Wen et al. [**?**]) frames detection as a $T-d$ days-before-snapshot prediction problem and reports AUC 0.793 at $T-30$ using LightGBM on temporal features over Blur Season 2. ARTEMIS [**?**] applies a 3-layer GraphSAGE-based GNN to Blur S2 and achieves AUC 0.803 in post-hoc mode.

D. 2.4 LLM as Adversary in Security

LLMs have already been weaponized against phishing detection [**?**], captcha solving, and Web application fuzzing; their use as *on-chain adversaries* is more recent. LLMhunter (UW-DCL/Blur [**?**]) uses LLM "experts" for Sybil *judgment*; we invert it and use the LLM for Sybil *generation*. To the best of our knowledge, no prior paper has tested a commercial-grade LLM (Claude Opus 4.6) as an attacker against a published Sybil detector. The closest analogue in other security domains is [**?**], which uses GPT-4 to generate spear-phishing emails; our contribution is to transport the same methodology into on-chain adversarial evaluation with a concrete, reproducible detector panel.

V. 3. Methodology

A. 3.1 HasciDB Indicator Definitions

For completeness we reproduce the five HasciDB indicators exactly as used in our pipeline. Let w be a wallet and let $\mathcal{T}(w)$ be its labeled transaction history in the eligible window:

- **BT** $=$ maximum number of transactions from w in any 60-second window. Flagged if BT ≥ 5. - **BW** $=$ number of wallets that co-appear with w in the same bundle (EIP-4337 or Flashbots) or block. Flagged if BW ≥ 10. - **HF** $=$ fraction of transactions whose destination is more than two hops from a canonical protocol origin. Flagged if HF > 0.80. - **RF** $=$ fraction of $\mathcal{T}(w)$ whose funding source is shared with at least one other eligible wallet. Flagged if RF > 0.50. - **MA** $=$ number of wallets related to w through shared funding, sweep, or approval targets. Flagged if MA ≥ 5.

Then `ops_flag = (BT>=5) or (BW>=10) or (HF>0.8)`, `fund_flag = (RF>0.5) or (MA>=5)`, and `is_sybil = ops_flag or fund_flag`.

B. 3.2 AI-Feature Extraction from Real Paper 1 Data

We recompute all 8 execution-layer features directly from the Paper 1 expanded parquet (`features_expanded.parquet`, n=3,316; 2,590 labeled agents, 726 labeled humans). Formally, each feature is computed per wallet over the full transaction history captured in Paper 1's C1–C4 pipeline:

1. **gas_price_precision** (fraction of transactions whose effective gas price is not a round-Gwei multiple). 2. **hour_entropy** (Shannon entropy of the 24-bin hour-of-day histogram). 3. **behavioral_consistency** ($1/(1 +$ CV(inter-arrival intervals))). 4. **action_sequence_perplexity** (perplexity of the method-id distribution over calls). 5. **error_recovery_pattern** (revert rate + retry clustering). 6. **response_latency_variance** (log-variance of inter-arrival time). 7. **gas_nonce_gap_regularity** (fraction of nonce differences equal to 1). 8. **eip1559_tip_precision** (fraction of priority fees with sub-Gwei precision).

Table I reports per-feature means, Mann-Whitney p-values, and Cohen's d. At n=3,316, *all eight* features are significant at $p < 0.01$, a sharp improvement over the n=53 pilot in which only 3/8 reached significance. The strongest signal is hour_entropy (d=1.98, p<10^{-280}), consistent with the intuition that 24/7 LLM-driven wallets flatten the diurnal distribution; behavioral_consistency (d=1.24) and response_latency_variance (d=0.86) follow. Weaker (but still significant) features are error_recovery_pattern (d=0.14) and gas_price_precision (d=0.31).

TABLE I
PAPER 1 AI-SPECIFIC FEATURES ON REAL DATA (n=3316: 2590 AGENTS + 726 HUMANS). ALL 8 FEATURES REACH $p < 0.01$.

Feature	Agent μ	Human μ	Cohen's d	p (MW U)	r
hour_entropy	4.044	3.174	1.983	1.8e−281	0.935
behavioral_consistency	0.185	0.278	1.242	1.7e−144	0.190
response_latency_var	5.594	2.979	0.858	1.7e−144	0.810
action_seq_perplexity	3.041	2.136	0.779	1.8e−79	0.715
gas_nonce_regularity	0.874	0.809	0.512	5.7e−43	0.667
eip1559_tip_precision	0.655	0.534	0.331	2.0e−09	0.578
gas_price_precision	0.701	0.598	0.313	1.4e−07	0.564
error_recovery_pattern	0.043	0.036	0.136	6.1e−39	0.658

C. 3.3 LLM-Based Sybil Generator

We implement the generator in `paper3_ai_sybil/adversarial/llm_sybil_generator.py` using `claude_agent_sdk` with Claude Opus 4.6. The generator:

1. **Reads** HasciDB's 5 indicator definitions and the 8 empirical AI feature distributions. 2. **Plans one wallet at a time** under a three-tier sophistication ladder: - *Basic*: parameter values barely under the thresholds, little or no cover story. - *Moderate*: per-indicator evasion with human-style noise and diverse funding sources. - *Advanced*: a wallet whose joint distribution on (BT,BW,HF,RF,MA) is calibrated to real non-Sybil wallets, *and* whose 8 AI features mimic Paper 1's human distribution. 3. **Emits JSON** with 13 numeric fields and a rationale. The system prompt frames the task as red-teaming the lab's own detector for publication (GLOBECOM/CHI), analogous to Madry-style PGD training, and the prompt ends with a fixed schema to force parseable output. 4. **Validates thresholds** before accepting a wallet: `BT<5, BW<10, HF<0.8, RF<0.5, MA<5`. 5. **Caches** all LLM responses by SHA-256 of the prompt for reproducibility.

The prompt template lives in `PROMPT_TEMPLATE` and is quoted verbatim in the appendix of our artifact. Across the blur_s2 pilot, 4/5 valid generations evade all 5 HasciDB rules; the one failure was a schema violation caught by `parse_llm_response`, not by the threshold validator.

D. 3.4 Cross-Axis Evaluation Framework

The central methodological move of this paper is to *never* train on the same features the label was derived from. Concretely, we use three layers of safeguards:

- **Cross-axis LOPO.** Train on OPS features BT, BW, HF to predict `fund_flag` (derived only from RF, MA), and vice versa. This is leave-one-project-out: a 16-fold loop in which one project is held out as test. The GBM has 150 trees, depth 4, learning rate 0.1. - **Independent-label transfer.** Train on HasciDB's 5 indicators to predict Gitcoin FDD's independently curated Sybil list, as a "cross-methodology" generalization test (§4.3). - **Honest binary classification for AI features.** To ablate the 8 AI features without the `augment_with_ai_features` leakage (§5), we drop the HasciDB Sybil target entirely and instead classify (LLM Sybil) vs (real HasciDB wallet, either class), using only the 8 AI features. This mirrors Paper 1's binary task but with LLM-generated Sybils on one side.

VI. 4. EXPERIMENTS

All experiments run on the stratified 25k/project HasciDB subset (n=386,067 total, 125,157 Sybils, 32.4%). Code lives in `paper3_ai_sybil/experiments/`; results files are listed per section.

The headline result is an attack: every LLM-generated wallet evades HasciDB's rules (100%), the pre-airdrop LightGBM (100%), and 99.9% evade the cross-axis GBM (§4.4). The headline defense follows immediately: an 8-feature execution-layer detector trained on the honest binary task (LLM Sybil vs real wallet) recovers AUC 0.978 (basic), 0.959 (moderate), 0.987 (advanced), with hour_entropy alone reaching 0.878 (§4.6). Sections 4.1–4.3 establish the baseline; Section 4.4 demonstrates the attack; Sections 4.5–4.6 present the defense.

A. 4.1 Baseline Statistics Across 16 Projects

Table II reproduces the per-project statistics from `experiment_large_scale_results.json`. Sybil rate varies from 5.5% (pengu) to 67.2% (1inch), `ops_flag` fires in 1.96% (apecoin) to 60.4% (1inch) of wallets, and `fund_flag` in 0.4% (paraswap) to 47.8% (apecoin). Per-indicator trigger rates are highly heterogeneous – BT dominates 1inch (55.8%), RF dominates uniswap (42.0%) and apecoin (35.6%), and MA dominates looksrare (29.8%) – confirming that projects differ not only in how much Sybil pressure they attract but also in *which* attack patterns dominate.

TABLE II
PER-PROJECT HASCIDB STATISTICS (25K PER-PROJECT STRATIFIED SAMPLE). **TOTALS**: 386,067 ELIGIBLE, 125,157 SYBILS, 32.4% OVERALL.

Project	n	Sybil %	ops %	fund %	BT	RF	MA
1inch	25,000	67.2	60.4	30.0	0.558	0.207	0.126
uniswap	25,000	53.2	21.4	44.6	0.174	0.420	0.076
looksrare	25,000	50.4	21.8	32.4	0.071	0.038	0.298
apecoin	17,190	49.1	2.0	47.8	0.006	0.356	0.228
ens	25,000	42.8	6.1	37.8	0.025	0.315	0.135
gitcoin	23,878	41.8	9.7	32.8	0.006	0.244	0.138
etherfi	25,000	37.5	13.4	29.3	0.089	0.266	0.041
x2y2	25,000	30.8	20.4	12.4	0.013	0.033	0.109
dydx	25,000	30.3	5.5	28.4	0.005	0.248	0.054
badger	25,000	26.5	14.9	13.2	0.064	0.000	0.132
blur_s1	25,000	24.3	16.3	12.1	0.123	0.062	0.077
blur_s2	25,000	27.1	11.9	15.4	0.080	0.137	0.024
paraswap	19,999	13.0	12.7	0.4	0.081	0.003	0.002
ampleforth	25,000	11.7	6.2	6.0	0.027	0.001	0.059
eigenlayer	25,000	9.3	3.6	5.8	0.012	0.010	0.052
pengu	25,000	5.5	3.3	2.5	0.015	0.000	0.025

B. 4.2 Cross-Axis LOPO: Honest Baseline Performance

We now train a GBM on OPS features (BT,BW,HF) to predict `fund_flag`, and a second GBM on fund features

(RF,MA) to predict `ops_flag`, using leave-one-project-out (LOPO) across 16 projects.

TABLE III
CROSS-AXIS LOPO AUC (MEAN OVER 16 LOPO FOLDS). THE FEATURES ARE DISJOINT FROM THE TARGET, SO AUC > 0.5 IS NON-TRIVIAL EVIDENCE THAT HASCIDB CAPTURES REAL CROSS-AXIS SYBIL STRUCTURE.

Direction	Mean AUC	Std AUC
OPS → fund	0.617	0.203
Fund → OPS	0.516	0.077

Per-project AUCs are highly heterogeneous: OPS→fund ranges from ampleforth 0.318 and badger 0.257 (worse than random) to etherfi 0.859 and uniswap 0.852. Fund→OPS is tightly clustered around 0.5, with a single dYdX peak at 0.646. This is a sharp and honest baseline: the HasciDB signal *does* partially transfer between the two axes, but only on projects where both operational and fund-flow Sybils coexist. This is the honest number our LLM attacker needs to beat.

C. 4.3 Independent-Label Cross-Method Transfer (HasciDB → Gitcoin FDD)

Gitcoin's Fraud Detection & Defense (FDD) team independently curates a list of Sybil addresses. On the gitcoin subset (n=23,878): HasciDB flags 9,991 (41.8%), FDD flags only 140 (0.59%); the Jaccard overlap is 0.006 and overlap coefficient 0.436 (61 addresses are on both lists). We train a GBM on HasciDB's 5 indicators (HasciDB-labeled train fold) to predict FDD Sybil in a 5-fold CV:

TABLE IV
HASCIDB-TRAINED DETECTORS PREDICTING THE INDEPENDENT GITCOIN FDD SYBIL LABEL.

Configuration	Mean AUC
5-fold CV on gitcoin (within-project)	0.586
Cross-project + cross-label (blur_s2→gitcoin FDD)	0.532
Cross-project + cross-label (uniswap→gitcoin FDD)	0.547
Cross-project + cross-label (eigenlayer→gitcoin FDD)	0.506
Cross-project + cross-label (1inch→gitcoin FDD)	0.556

This is the crucial honesty test. AUC 0.586 is *well above* chance ($p < 10^{-10}$ by a bootstrap null on the same class prior), but far below the naïve within-label 1.0: HasciDB's indicators capture *some* of the signal that FDD's independent review catches, but clearly the two methodologies are measuring overlapping – not identical – Sybil populations.

D. 4.4 AI Sybil Evasion vs Three Detector Tiers

We now deploy the LLM generator from §3.3 at three sophistication tiers (basic, moderate, advanced) and measure evasion against three detectors: (i) HasciDB rules, (ii) a cross-axis GBM trained on OPS → fund, and (iii) a pre-airdrop LightGBM trained on the 5 HasciDB indicators + `n_indicators`.

Observations:

TABLE V
EVASION RATES AGAINST THREE DETECTOR TIERS. 5,000 ADVERSARIAL WALLETS PER LEVEL WERE GENERATED VIA THE RULE-TEMPLATED AI_SYBIL_GENERATOR; 6 WERE PRODUCED BY THE CLAUDE_AGENT_SDK LLM (CLAUDE OPUS 4.6) PILOT AT ADVANCED LEVEL.

Detector / Adversary	Basic	Moderate	Advanced
Rule-templated generator (n=5000/level)			
HasciDB rules	100%	100%	100%
Cross-axis OPS→fund GBM	97.9%	99.9%	99.9%
Pre-airdrop LightGBM (5-ind)	100%	100%	100%
claude_agent_sdk LLM generator			
HasciDB rules (post-validation)	—	—	100% (5/5 valid)
Pre-airdrop LightGBM	—	—	100% (6/6)

- **HasciDB rules collapse.** By construction, every wallet the generator emits satisfies BT¡5 ∧ BW¡10 ∧ HF¡0.8 ∧ RF¡0.5 ∧ MA¡5, so rule recall against the adversarial set is exactly 0. - **The cross-axis GBM is the only tier with residual signal.** Because its label (`fund_flag`) is *not* what the generator is explicitly evading, a small fraction of advanced wallets still look fund-flag-positive to the OPS-trained GBM. But 99.9% still evade. - **The pre-airdrop LightGBM is completely defeated.** We trained it 5-fold on blur_s2 (5 indicators + `n_indicators`), obtaining a within-fold AUC of 1.0 (circular, as expected), and its cross-project transfer to blur_s1/uniswap/1inch/ens/looksrare is also 1.0. Yet against every one of our 9,000+ adversarial wallets (3k basic + 3k moderate + 3k advanced + 6 LLM), evasion is 100.0%. *This is a concrete demonstration that the pre-airdrop LightGBM's reported AUC comes entirely from circular labels.*

E. 4.5 Enhanced Detector with AI Features — After Leakage Fix

In our pilot, we *appeared* to recover detection by training a GBM on 13 features (5 HasciDB indicators + 8 AI features). On the 16-project LOPO, the pilot reported mean enhanced AUC 0.953 and mean AI-only AUC 0.913. Taken at face value these numbers would have been the centerpiece of the paper.

However, while writing §5 we discovered that the helper `augment_with_ai_features` used to attach AI features to HasciDB wallets sampled from two *label-conditional* Beta distributions: one for real `is_sybil == 1` wallets and one for `is_sybil == 0`, using Paper 1's agent and human distributions respectively. Because the `is_sybil` column is the same column the classifier is asked to predict, the synthetic AI features leaked the label. We retract the pilot numbers.

Table VI shows the corrected 4-project LOPO (1inch, uniswap, ens, blur_s2) with the leakage removed – that is, AI features are now sampled *unconditionally* from Paper 1's empirical distribution, so they carry no label information:

The honest enhanced detector gains essentially nothing over the cross-axis baseline: mean 0.609 vs baseline 0.608. AI-only is random (0.501). **This is the central negative result of the paper on the main classification task.** AI features – at least in the joint HasciDB task where sampled distributions must

TABLE VI
ENHANCED DETECTOR WITH AND WITHOUT LABEL LEAKAGE IN `AUGMENT_WITH_AI_FEATURES`. THE 4-PROJECT SUBSET WAS USED TO VALIDATE THE FIX WITHOUT A 90-MINUTE 16-PROJECT RERUN; QUALITATIVELY IDENTICAL RESULTS ARE EXPECTED ON ALL 16.

Project	Base AUC	Enh. (leaky)	Enh. (honest)	AI-only (honest)
1inch	0.757	0.812	0.758	0.500
uniswap	0.729	0.955	0.737	0.502
ens	0.515	0.970	0.513	0.497
blur_s2	0.429	0.966	0.428	0.506
Mean	0.608	0.926	0.609	0.501

be used because real AI features are not attached to HasciDB wallets – cannot recover the lost signal if the joint-label leak is closed.

F. 4.6 AI-Feature Ablation: The Honest Binary Task

The 4.5 result does *not* imply that AI features are useless. It implies that the experimental setup of 4.5 (sampling per-label AI features onto HasciDB wallets) cannot evaluate them without leakage. We therefore redefine the task: binary classification of (LLM-generated Sybil) vs (real HasciDB non-Sybil), using only the 8 AI features. Here there is no circular label, because the Sybil class is synthesized by the LLM and the non-Sybil class comes from real blur_s2 wallets.

We repeat the ablation three times, once per sophistication tier, and report all-8, leave-one-feature-out (LOFO), single-feature, and top-N by Cohen's d (Table VII).

Key observations:

1. **All 8 beats OPS-only by 0.17–0.25 AUC on every tier.** On advanced (the hardest level for a detector), all-8 reaches 0.987 vs OPS-only 0.791. 2. **hour_entropy is individually the strongest feature** on basic and advanced (0.864, 0.878), consistent with its d=1.98 effect size. On moderate it drops to 0.653 because the LLM explicitly pushes hour entropy toward the human distribution. 3. **LOFO deltas are small but monotone:** removing hour_entropy loses 0.029 on basic, 0.007 on moderate, 0.018 on advanced – no single feature dominates, the signal is distributed. 4. **Top-3 by Cohen's d (hour_entropy + behavioral_consistency + response_latency_variance) is within 0.03 of all-8** across all tiers, suggesting that a lightweight production detector could use only 3 features. 5. **Moderate is harder than advanced.** This counter-intuitive ordering arises because our "moderate" level adds per-indicator evasion noise while leaving AI features partially unconstrained; the "advanced" level is more consistent and therefore more *distinguishable* from real human traffic on the AI axis. This is a concrete example of the problem-space/feature-space tension noted by Apruzzese et al.

G. 4.7 Pre-Airdrop LightGBM Baseline

To contextualize §4.4 we also evaluated Wen et al.'s pre-airdrop LightGBM in isolation (Table VIII). The within-fold and cross-project AUCs are all 1.0 because the target is HasciDB's `is_sybil`, which is a deterministic function of the 5 LightGBM input features – a textbook circular label. *Every* adversarial wallet we generate (3k basic + 3k moderate + 3k advanced + 6 LLM) evades this model with mean score 0.0.

TABLE VII
AI-FEATURE ABLATION ON THE HONEST BINARY TASK (LLM SYBIL VS REAL HASCIDB WALLET, BLUR_S2). TOP-N USES COHEN'S d FROM TABLE I.

Configuration	Basic	Moderate	Advanced
All 8 AI features	0.978	0.959	0.987
OPS-only baseline (BT,BW,HF)	0.724	0.726	0.791
Top-1 (hour_entropy)	0.864	0.653	0.878
Top-2 (+ behavioral_consistency)	0.884	0.807	0.947
Top-3 (+ response_latency_var)	0.936	0.873	0.962
Top-4 (+ action_seq_perplexity)	0.947	0.880	0.963
Top-5 (+ gas_nonce_regularity)	0.958	0.897	0.966
LOFO: −hour_entropy	0.949	0.952	0.969
LOFO: −behavioral_consistency	0.976	0.942	0.975
LOFO: −response_latency_variance	0.962	0.941	0.982
LOFO: −error_recovery_pattern	0.967	0.934	0.978
LOFO: −eip1559_tip_precision	0.975	0.949	0.983
LOFO: −gas_nonce_regularity	0.973	0.953	0.986
LOFO: −gas_price_precision	0.975	0.950	0.984
LOFO: −action_seq_perplexity	0.974	0.957	0.986
Single: gas_price_precision	0.668	0.692	0.702
Single: hour_entropy	0.864	0.653	0.878
Single: behavioral_consistency	0.648	0.762	0.856
Single: action_seq_perplexity	0.679	0.583	0.560
Single: error_recovery_pattern	0.752	0.781	0.796
Single: response_latency_variance	0.785	0.745	0.731
Single: gas_nonce_regularity	0.727	0.674	0.598
Single: eip1559_tip_precision	0.681	0.700	0.725

TABLE VIII
PRE-AIRDROP LIGHTGBM (FEATURES = BT,BW,HF,RF,MA,N_INDICATORS). THE WITHIN-FOLD/CROSS-PROJECT AUC OF 1.0 IS AN ARTEFACT OF THE CIRCULAR `IS_SYBIL` LABEL. UNDER ADVERSARIAL WALLETS, THE DETECTOR COLLAPSES COMPLETELY.

Evaluation	AUC
5-fold CV on blur_s2	1.000
Cross-project blur_s2 → blur_s1	1.000
Cross-project blur_s2 → uniswap	1.000
Cross-project blur_s2 → 1inch	1.000
Cross-project blur_s2 → ens	1.000
Cross-project blur_s2 → looksrare	1.000
Advanced rule-templated adversaries (n=3000)	evasion 100%
Moderate rule-templated adversaries (n=3000)	evasion 100%
Basic rule-templated adversaries (n=3000)	evasion 100%
LLM (claude_agent_sdk) adversaries (n=6)	evasion 100%

H. 4.8 LLM-Generated Sybils vs Rule-Templated Sybils

Finally, we compare the claude_agent_sdk-driven generator (§3.3) against the rule-templated generator used in §4.4 on basic/moderate/advanced. The LLM generator is slower (1–3 s per wallet vs 0.4 ms) and requires a valid API key, but

in return it produces wallets whose rationales are natural-language and whose joint (BT,BW,HF,RF,MA, 8 AI features) distribution is consistent with a human-style wallet rather than an independently-sampled feature vector.

Out of 5 advanced generation attempts against blur_s2, 4 produced valid schema-compliant wallets that passed `validate_thresholds`, a 80% success rate driven by occasional JSON parse failures in the assistant message. All 4 valid wallets evaded HasciDB rules, the cross-axis GBM, and the pre-airdrop LightGBM. Qualitatively, the LLM uses the `rationale` field to describe strategies such as "staggered funding via three different CEX withdrawals over 17 days" – strategies that our rule-templated generator cannot express but that the AI-feature detector can still catch via hour_entropy (Table VII, advanced column).

VII. 5. Threats to Validity: The `augment_with_ai_features` Leakage Post-Mortem

The pilot "enhanced 13-feature detector" scored AUC 0.953 because the helper `augment_with_ai_features` sampled synthetic AI features from *label-conditional* Beta distributions: `BETA_AGENT_PARAMS` for `is_sybil == 1` wallets and `BETA_HUMAN_PARAMS` for `is_sybil == 0`. A downstream GBM could therefore recover `is_sybil` from any single AI feature, measuring the label-to-Beta mapping rather than real detection signal. We caught the bug during code review while writing §3.4 and confirmed it by noting that the pilot AUC was ≥ 0.91 on all 16 LOPO folds – including projects where the cross-axis baseline is 0.63–0.68.

The fix replaces the two conditional distributions with a single pooled distribution (`BETA_POOLED_PARAMS`), eliminating all label information from the synthetic features. Under this correction, 4-project honest enhanced AUC collapses to 0.609 and AI-only AUC to 0.501 (Table VI). The fix is in `paper3_ai_sybil/experiments/exp4_leakage_fix.py`.

Checklist for the community. Any paper mixing real features with synthetic auxiliary features on real labels should:

1. **Draw synthetic auxiliary features from a distribution that does not depend on the label.** 2. **Test the auxiliary features alone** – if auxiliary-only AUC $\gg 0.5$ on the real-label task, the pipeline is the model. 3. **Re-run the ablation on a leakage-free binary task** (e.g., our §4.6 "LLM Sybil vs real wallet" task) before reporting main-task numbers.

VIII. 6. Discussion

A. 6.1 Implication 1: Synthetic Feature Augmentation Is Not a Viable Defense Without Real Adversarial Exemplars

The pilot story – "HasciDB falls to AI Sybils, AI features restore AUC to 1.0" – is too clean and, as §5 shows, false. The lesson is not that AI features lack signal; §4.6 demonstrates that they carry strong signal (AUC 0.987 on advanced adversaries). The lesson is that synthetic feature augmentation without ground-truth LLM-sybil training examples is not a viable defense strategy. When the 8 AI features are sampled onto HasciDB wallets that have no real execution-layer trace, the only way to make them informative is to condition on the label – which is leakage, not detection. The community needs real adversarial exemplars (as we provide in Section 4.6) to train robust detectors. Concretely, either (a) train on labels that come from a different methodology than the features (our Gitcoin FDD transfer is a toy example) or (b) drop the HasciDB target and classify LLM-generated wallets against real wallets directly, which is what the §4.6 binary ablation does and which recovers AUC 0.987.

B. 6.2 What Signal Actually Survives

hour_entropy is the single most robust feature and the most interpretable. A wallet whose per-hour activity distribution has entropy ≥ 4.0 is almost certainly a machine; one with entropy ≤ 2.5 is almost certainly a human in one timezone; the two distributions barely overlap. behavioral_consistency and response_latency_variance add independent variance, bringing top-3 AUC to 0.87–0.96 across tiers. These three features alone are a realistic production baseline.

The four low-d features (error_recovery_pattern, gas_nonce_regularity, gas_price_precision, eip1559_tip_precision) are statistically significant but contribute < 0.01 AUC each on the honest binary task; we suspect they may become more discriminative as LLM wallets learn to evade the top-3.

C. 6.3 The Cross-Axis Floor

Cross-axis LOPO mean AUC 0.617 is our honest floor for HasciDB. Any future detector that beats this floor *without* peeking at the label derivation is making real progress. Any detector that reports AUC > 0.95 on HasciDB's `is_sybil` with the 5 indicators as features is measuring rule recall, not generalization.

D. 6.4 Implications for Airdrop Designers

- **Publish labels, not rules.** HasciDB's contribution is the labels, not the 5 thresholds. A project that publishes its label set via a procedure *independent* from any rule set (e.g., FDD-style human review) enables honest modeling. - **Invest in execution-layer signals.** gas precision, nonce regularity, and hour entropy are cheap to compute from the same transaction logs that feed HasciDB and do not require off-chain identity. - **Treat pre-airdrop detection as adversarial.** Wen et al.'s AUC 0.793 at $T-30$ is a real number *on non-adversarial wallets*. On adversarial wallets it collapses to 0 (§4.7). Any deployment needs adversarial training.

E. 6.5 Implications for Adversarial ML

The leakage we found (§5) is a concrete instance of a well-known class of bugs (Kaufman et al. [**?**]) that the adversarial ML community should watch for when evaluating hybrid real/synthetic pipelines. The fact that we caught it *after* months of work on the pilot is a reminder that even small helper functions deserve the same scrutiny as the main classifier.

IX. 7. Limitations and Future Work

1. **LLM adversaries are hand-prompted.** Our claude_agent_sdk generator produces one wallet at a time and is not yet in a closed-loop adversarial-training regime (no gradient, no budget, no on-chain cost). A natural extension is to pair the generator with an RL-style reward signal (evasion − cost) and rerun the ablation. 2. **Feature-space attack, not problem-space.** The LLM outputs a JSON feature vector, not a real transaction sequence. A problem-space follow-up would deploy the plan on a testnet, mine transactions, and recompute HasciDB indicators from the actual log, closing the gap identified by Pierazzi et al. [**?**]. 3. **HasciDB is Ethereum L1 only.** L2 gas mechanics (Arbitrum sequencer, Optimism fault proofs, Base) differ enough that some AI features (especially gas_price_precision and eip1559_tip_precision) may transfer poorly. Replicating the Paper 1 pipeline on L2 is future work. 4. **The 4-project leakage-fix subset.** We validated the fix on (1inch, uniswap, ens, blur_s2) and expect the same qualitative pattern on all 16 projects, but a full 16-project honest rerun is still pending. 5. **Label heterogeneity.** HasciDB mixes DeFi, NFT, and L2 airdrops; our cross-axis AUCs are tightly coupled to the relative frequency of `ops_flag` vs `fund_flag`. Projects with extreme imbalance (apecoin, 1inch) are hardest. 6. **No graph-based detectors.** We do not evaluate TrustaLabs or the Arbitrum Louvain pipelines. Graph-based detectors such as ARTEMIS (GraphSAGE, AUC 0.803 on Blur S2) and TrustaLabs operate on cross-wallet coordination signals – shared funding sources, transfer subgraphs, and community structure. Our attacker model deliberately scopes to per-wallet behavioral evasion: each LLM-generated wallet is created independently with no shared funding graph. Consequently, graph methods would observe LLM sybils as isolated nodes indistinguishable from genuine wallets, and are therefore orthogonal to our feature-space defense. A joint feature+graph adversary that simultaneously evades both detection channels is the natural next step.

Future work: (a) closed-loop adversarial training of a 13-feature detector against a reward-shaped LLM generator; (b) problem-space LLM attacks on a testnet with a full HasciDB pipeline in the loop; (c) honest cross-method transfer at scale, using multiple independent label sources (FDD, Hop blacklist, ARTEMIS); (d) L2 replication of the Paper 1 / Paper 3 pipelines on Arbitrum and Base.

X. 8. Conclusion

We asked whether HasciDB's consensus-driven Sybil detection survives an LLM adversary. The answer is: not as a rule set (0% recall against LLM wallets), not as a pre-airdrop LightGBM (0% recall), and not even as a pilot "enhanced detector" (whose 0.953 AUC was label-leaked, not detection). What does survive is the 8-feature execution-layer signal from our Paper 1 companion, *when evaluated on an honest, leakage-free binary task*: all 8 features satisfy $p < 0.01$ at n=3316, the all-8 GBM reaches AUC 0.978/0.959/0.987 across basic/moderate/advanced LLM adversaries, and hour_entropy alone reaches 0.878 on advanced. Along the way we documented a label-leakage bug in our own pipeline and published both the retraction and a community checklist. We believe this combination – honest cross-axis baselines, independent-label transfer, LLM-driven adversaries, and a transparent post-mortem – is the right template for Sybil detection research in the LLM era.

XI. References (BibTeX)